

GUIÓN DE PRESENTACIÓN

Generador Académico RAG

Speech seccionado por integrante para la exposición. Cada parte sirve también como material de estudio: lo que decís y lo que te pueden preguntar.

Grupo 14 · Ciencia de Datos · UTN FRLP

Martin, Tomás · Gentil, Mora · Natalichio, Santiago · Cuenca, Juan Bautista · Ocampo, Simón · Aubert, Lautaro

6 turnos · 10 diapositivas · Duración objetivo: 18 a 24 minutos

1

Martin, Tomás

Apertura + problema

Slides 1-2

≈ 3 min

SLIDE 1 · APERTURA

Buenas a todos. Somos el Grupo 14 y nuestro proyecto se llama **Generador Académico RAG**. Arranco con una pregunta: [pausa] ¿qué pasa si le pedimos a una IA "armame el plan del próximo cuatrimestre"? Si solo sabe lo genérico de internet, no puede: no conoce **tus** notas, **tus** correlativas, **tu** avance. Eso es justo lo que resolvimos.

SLIDE 2 · EL PROBLEMA

Y acá la primera aclaración importante, porque es la crítica que nos íbamos a comer: "che, eso lo hace un SQL". Y es verdad... **si** el producto fueran consultas tipo "¿qué nota tengo en Bases de Datos?". Eso es **lookup**, buscar un dato, no es IA generativa.

Nuestro reencuadre fue ese: el producto no son respuestas, son **artefactos nuevos generados** (un plan justificado, un informe, una carta de pasantía). El SQL te dice qué **podés** cursar; nuestro sistema te dice qué **conviene** y por qué, redactado. Y tampoco es NotebookLM: la gracia del TP es que el pipeline RAG lo **construimos nosotros** y lo mostramos. Mis compañeros lo van desarmando.

CLAVES PARA ESTUDIAR / SI TE PREGUNTAN

- **Qué hace:** genera artefactos académicos (planes, informes, cartas) anclados en el historial real del grupo y el plan de estudios.
- **"Lo hace un SQL":** no, un SQL hace lookup; nosotros generamos texto nuevo con razonamiento y tono.
- **"Es NotebookLM":** no, construimos el pipeline RAG (chunking, embeddings, vector DB, retrieval) en vez de usar una caja negra.

Cierre: "Para generar bien, primero hay que tener buenos datos. Mora les cuenta con qué trabajamos." → pasa a **Mora**.

SLIDE 3 · DATOS Y EXPLORACIÓN

Gracias, Tomás. El corazón del proyecto son los datos, y son **privados**: el modelo no los conoce. Parseamos PDFs reales —los estados académicos y exports de notas de **los seis integrantes** del grupo, más el Plan de Sistemas 2023 con todas las correlatividades, más el Diseño Curricular completo— y obtenemos **413 filas + 136 documentos**.

Lo interesante es que el corpus es **mixto**, y por eso usamos **dos bases de datos**: lo **tabular** (cada materia con su nota y su año, y las correlatividades) va en una base **relacional** —ahí un lookup o un join exacto es lo correcto—; lo **no estructurado** (el texto del plan) va en una base **vectorial**, para búsqueda semántica. El desafío técnico más grande fue el **join**: los códigos de materia no coinciden entre el estado y el plan, así que unimos por el **nombre normalizado**, sin tildes ni mayúsculas. Parseamos las 36 obligatorias con sus correlativas.

Sobre eso hicimos el EDA: son **221 notas reales** de los seis, con promedios parejos entre **8.0 y 8.7**. Las diferencias verdaderas aparecen al desagregar por área temática, y eso es justo lo que después explota el diagnóstico grupal.

CLAVES PARA ESTUDIAR / SI TE PREGUNTAN

- **Corpus:** 413 filas a la relacional (377 de estados + 36 correlativas) + 136 docs a la vectorial (100 chunks de prosa del diseño + 36 fichas de materia con contenidos).
- **Por qué dos bases:** los datos de alumno e historial son tabulares → relacional (lookup/join exacto); la documentación del plan es texto largo → vectorial (semántico). El agente combina ambas.
- **Por qué normalizar:** los códigos institucionales no coinciden entre documentos; el nombre normalizado es la clave de join.
- **EDA:** 221 notas reales de los 6 integrantes, promedios 8.0–8.7; se compara por área temática.
- **Equivalencias 2008→2023:** el export de notas usa nombres del Plan 2008; los canonizamos al 2023 (Anexo II) para no perder notas por equivalencia.
- **Export de notas más completo:** trae materias aprobadas que no figuran en el estado académico; se suman como aprobadas con su nota (si no, el plan las recomendaría como pendientes).

Cierre: "Con esos datos limpios, Santi les muestra cómo los convertimos en algo que el modelo puede consultar." → pasa a **Santiago**.

SLIDE 4 · EL PIPELINE RAG

Gracias, Mora. Acá está el núcleo del trabajo. RAG significa **Retrieval-Augmented Generation**: en vez de pedirle al modelo que sepa todo, le **recuperamos** el contexto justo y se lo damos para que genere sobre eso.

El flujo: los PDFs pasan por **pdfplumber** a texto y los partimos en dos. Los datos **tabulares** —historial y correlatividades— van a una base **relacional (SQLite)**. La **prosa del plan** la vectorizamos con **MiniLM** y la guardamos en **Chroma**, la base vectorial. Cuando llega un pedido, recuperamos de **ambas** y se lo pasamos a **qwen2.5 (14B)** vía Ollama. Todo el pipeline corre local; la generación la podemos correr en una **GPU gratuita de Colab** para usar un modelo grande en segundos, o 100% local. Los datos nunca pasan por un proveedor externo: la privacidad se mantiene.

Un detalle que nos importó: el retrieval es **híbrido** y combina las dos bases. La **relacional**, con SQL, ancla exactamente las materias aprobadas de un alumno y sus correlativas —eso garantiza que no se inventen notas—. La **vectorial**, por similitud de embeddings, trae la documentación del plan para lenguaje natural. Una misma consulta cruza las dos en un solo contexto.

SLIDE 5 · EL ESPACIO SEMÁNTICO

¿Y cómo sabemos que los embeddings funcionan? Proyectamos las 36 materias con **t-SNE** y se ve que las de la misma área caen juntas. Eso valida que el espacio capturó la estructura del plan, y es lo que habilita el recomendador por **matching semántico**: cruzar dónde te fue bien con materias afines. Eso un SQL no lo puede hacer; necesita similitud sobre texto.

CLAVES PARA ESTUDIAR / SI TE PREGUNTAN

- **RAG:** recuperar contexto relevante y dárselo al modelo para que genere anclado, en vez de confiar en su memoria.
- **Persistencia híbrida:** SQLite (relacional) para lo tabular; Chroma (vectorial, all-MiniLM-L6-v2 local) para la documentación del plan.
- **Retrieval híbrido:** SQL exacto sobre la relacional (grounding de notas/correlativas) + similitud sobre la vectorial (texto del plan); se combinan.
- **t-SNE:** valida que materias de la misma área quedan cercanas en el espacio.

Cierre: "Ya tenemos el contexto. Juan les muestra qué hacemos con él: generar." → pasa a **Juan Bautista**.

4

Cuenca, Juan Bautista

Generación • prompting • tonos

Slide 6

≈ 3 min

SLIDE 6 • UN DATO, TRES TONOS

Gracias, Santi. Tenemos seis generadores: plan de cursada, informe de trayectoria, carta de pasantía, recomendación de orientación, diagnóstico grupal y simulación what-if. Todos comparten la misma regla de prompting: el modelo usa **exclusivamente** el contexto recuperado y tiene prohibido inventar notas o materias.

Pero lo que lo hace **generativo** de verdad es el **control de tono**. Miren la slide: el mismo dato, "podés cursar Ciencia de Datos", dicho en tres registros. En **técnico** es seco y preciso. En **motivacional** te destaca el logro. En **honesto** te marca también la debilidad sin vueltas.

Los datos son idénticos, salen del mismo contexto. Lo único que cambia es cómo se redacta. Eso es exactamente lo que un reporte fijo o un SQL no pueden darte: es síntesis de texto, no una consulta.

CLAVES PARA ESTUDIAR / SI TE PREGUNTAN

- **6 artefactos:** plan, informe, carta, orientación, diagnóstico grupal, what-if.
- **Anti-alucinación:** el system prompt obliga a usar solo el contexto y a no inventar.
- **Tono:** técnico / motivacional / honesto sobre los mismos datos; es un control de producto, no estético.

Cierre: "¿Y cómo demostramos que esto realmente usa los datos y no chamuya? Simón lo mide en vivo." → pasa a **Simón**.

5

Ocampo, Simón

Demo con/sin RAG · evaluación

Slides 7-8

≈ 4 min

SLIDE 7 · CON RAG VS. SIN RAG (DEMO EN VIVO)

Gracias, Juan. Esta es nuestra demo estrella, y la voy a correr en vivo en la web app. El mismo pedido, un plan de cursada, con un botón: **con RAG** y **sin RAG**.

Con RAG, miren la consola: a la izquierda aparece el contexto recuperado —las materias y notas reales que vienen de la base relacional, más la documentación del plan de la vectorial—, y a la derecha el modelo genera el plan citando esos datos. Sin RAG, le sacamos el contexto: el modelo no tiene de dónde agarrarse, así que se generaliza o directamente inventa materias que no existen.

SLIDE 8 · CÓMO LO MEDIMOS

Y esto no es impresión nuestra, lo **medimos** con cuatro métricas en tres ejes. Las dos de **grounding**, léxico y semántico, miden cuánto se solapa la salida con el contexto. Pero la más contundente es la **precisión factual**: agarramos cada par materia-nota que el modelo afirma y lo verificamos contra el registro real. Con RAG da **1.0** —cero notas inventadas—; sin RAG no cita un solo dato real. Y sumamos **faithfulness**, donde el propio modelo juzga la fidelidad al contexto: **0.73 con RAG contra 0.50 sin**. En las cuatro, con RAG le gana a sin RAG.

CLAVES PARA ESTUDIAR / SI TE PREGUNTAN

- **Demo:** mismo input, con/sin contexto; muestra que el valor está en el dato privado.
- **4 métricas / 3 ejes:** solapamiento (léxico, semántico), corrección factual (precisión) y juicio de un LLM (faithfulness).
- **Grounding léxico/semántico:** tokens de la salida en el contexto / coseno de embeddings; el semántico reconoce paráfrasis que el léxico penaliza.
- **Precisión factual (la estrella):** de los pares (materia, nota) que afirma, qué fracción coincide con el registro real. Con RAG = **1.0** (cero inventadas); sin RAG no cita ninguno.
- **Faithfulness:** un LLM puntúa 0–1 la fidelidad al contexto. **0.73 con / 0.50 sin.**
- **Resultado (4 artefactos, qwen2.5:14B):** en las 4 métricas con RAG > sin RAG. Grounding léxico 0.55/0.36, semántico 0.57/0.51.
- **Honestidad metodológica:** el léxico varía entre corridas (generación estocástica); lo robusto es la dirección y la precisión 1.0. Ninguna métrica detecta contradicciones lógicas sutiles.
- **Si falla la demo en vivo:** mostrar la celda equivalente del notebook.

Cierre: "Más allá de un alumno, esto sirve para todo el grupo. Lautaro cierra con eso." → pasa a **Lautaro**.

SLIDE 9 · DIAGNÓSTICO GRUPAL

Gracias, Simón. Para cerrar el lado de análisis: aplicamos **KMeans** sobre el perfil por área de cada integrante y lo proyectamos con PCA. Eso agrupa a la gente por fortalezas afines, y el generador de diagnóstico grupal usa esos clusters para **repartir roles** de un proyecto, con justificación. Es el pipeline de Ciencia de Datos completo: EDA, visualización, clustering, todo conectado al RAG.

SLIDE 10 · CONCLUSIONES

¿Qué demostramos? Cuatro cosas. Una: construimos un pipeline RAG completo, no una caja negra. Dos: el sistema **genera** artefactos con tono, no responde preguntas. Tres: la demo con/sin RAG hace **medible** el valor del conocimiento privado. Y cuatro: hicimos el pipeline de DS entero, de los datos crudos a la evaluación.

Todo esto está en el notebook, el informe, y la web app que vieron funcionando. Muchas gracias. Quedamos para las preguntas.

CLAVES PARA ESTUDIAR / SI TE PREGUNTAN

- **Clustering:** KMeans sobre perfil por área (estandarizado), PCA para visualizar.
- **Cierre del valor:** pipeline propio + generación + evaluación medible + DS completo.
- **"¿Para qué sirve fuera del TP?":** cualquier estudiante podría generar su plan/carta/informe sobre su propio historial.

Fin de la exposición. Reparto de preguntas: técnicas de pipeline → Santiago; datos → Mora; evaluación → Simón.